AIAC ASSIGNMENT – 11.3

NAME: T.Prashanthi

2303A52495

BATCH: 50

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).

2. Implement the same functionality using a linked list for dynamic memory allocation.

3. Implement the following operations in both approaches:

o Add a contact

o Search for a contact

o Delete a contact

4. Use GitHub Copilot to assist in generating search and delete methods.

5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency

o Deletion efficiency

Expected Outcome

- **Two working implementations (array-based and linked-list-based).**
- **A brief comparison explaining performance differences.**

PROMPT:

Build a Smart Contact Manager using two approaches:Array (Python list),Linked List

#Implement add_contact(name, phone),Implement search_contact(name),Implement delete_contact(name)

For Linked List:Create a Node class,Use dynamic memory (next pointer)

#Both implementations support all operations correctly,Return "Not Found" if contact does not exist

#Compare array vs linked list based on:insertion efficiency and deletion efficiency

Generate clean, readable Python code with example test cases

CODE:

```python
# -------- ARRAY IMPLEMENTATION --------
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print("Contact added")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                print("Found:", contact["name"], "-", contact["phone"])
                return
        print("Contact not found")

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                self.contacts.remove(contact)
                print("Contact deleted")
                return
        print("Contact not found")

    def display_contacts(self):
        print("\nArray Contacts:")
        for contact in self.contacts:
            print(contact["name"], "-", contact["phone"])
```

```python
# -------- LINKED LIST IMPLEMENTATION --------
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node
        print("Contact added")

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                print("Found:", temp.name, "-", temp.phone)
                return
            temp = temp.next
        print("Contact not found")
```

```python
    def delete_contact(self, name):
        temp = self.head
        prev = None
        while temp:
            if temp.name == name:
                if prev is None:
                    self.head = temp.next
                else:
                    prev.next = temp.next
                print("Contact deleted")
                return
            prev = temp
            temp = temp.next
        print("Contact not found")

    def display_contacts(self):
        print("\nLinked List Contacts:")
        temp = self.head
        while temp:
            print(temp.name, "-", temp.phone)
            temp = temp.next


# -------- TESTING --------
print("ARRAY IMPLEMENTATION")
array_manager = ContactManagerArray()
array_manager.add_contact("Rahul", "9876543210")
array_manager.add_contact("Anita", "9123456780")
array_manager.display_contacts()
array_manager.search_contact("Rahul")
array_manager.delete_contact("Rahul")
array_manager.display_contacts()
```

```python
print("\nLINKED LIST IMPLEMENTATION")
linked_manager = ContactManagerLinkedList()
linked_manager.add_contact("Rahul", "9876543210")
linked_manager.add_contact("Anita", "9123456780")
linked_manager.display_contacts()
linked_manager.search_contact("Anita")
linked_manager.delete_contact("Rahul")
linked_manager.display_contacts()
```

OUTPUT:

```
11.3.py
ARRAY IMPLEMENTATION
Contact added
Contact added

Array Contacts:
Rahul - 9876543210
Anita - 9123456780
Found: Rahul - 9876543210
Contact deleted

Array Contacts:
Anita - 9123456780

LINKED LIST IMPLEMENTATION
Contact added
Contact added

Linked List Contacts:
Rahul - 9876543210
Anita - 9123456780
Found: Anita - 9123456780
Contact deleted

Linked List Contacts:
Anita - 9123456780
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

- **Working queue and priority queue implementations.**
- **Correct prioritization of faculty requests.**

Prompt:

Generate a python code to Build a Library Book Request System using:Queue (FIFO),Priority Queue (faculty > student)

#Implement enqueue() and dequeue() methods,Queue should process requests in FIFO order,Priority Queue should prioritize faculty over students

#Each request should include name and type (student/faculty),Test with mixed requests,Print processed order

CODE:

```python
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, name, user_type):
        self.queue.append({"name": name, "type": user_type})
        print(f"Request added: {name} ({user_type})")

    def dequeue(self):
        if not self.queue:
            print("No requests to process")
            return None
        request = self.queue.pop(0)
        print(f"Processing request: {request['name']} ({request['type']})")
        return request

class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, name, user_type):
        self.queue.append({"name": name, "type": user_type})
        print(f"Request added: {name} ({user_type})")

    def dequeue(self):
        if not self.queue:
            print("No requests to process")
            return None
        # Prioritize faculty over students
        for i, request in enumerate(self.queue):
            if request["type"] == "faculty":
                print(f"Processing request: {request['name']} ({request['type']})")
                return self.queue.pop(i)
        # If no faculty requests, process the first student request
        request = self.queue.pop(0)
        print(f"Processing request: {request['name']} ({request['type']})")
        return request
```

```python
# Testing the Queue and PriorityQueue
print("QUEUE (FIFO) IMPLEMENTATION")
fifo_queue = Queue()
fifo_queue.enqueue("Alice", "student")
fifo_queue.enqueue("Bob", "faculty")
fifo_queue.enqueue("Charlie", "student")
fifo_queue.dequeue()
fifo_queue.dequeue()
fifo_queue.dequeue()
print("\nPRIORITY QUEUE (Faculty > Student) IMPLEMENTATION")
priority_queue = PriorityQueue()
priority_queue.enqueue("Alice", "student")
priority_queue.enqueue("Bob", "faculty")
priority_queue.enqueue("Charlie", "student")
priority_queue.dequeue()
priority_queue.dequeue()
priority_queue.dequeue()
```

OUTPUT:

```
11.3.py
QUEUE (FIFO) IMPLEMENTATION
Request added: Alice (student)
Request added: Bob (faculty)
Request added: Charlie (student)
Processing request: Alice (student)
Processing request: Bob (faculty)
Processing request: Charlie (student)

PRIORITY QUEUE (Faculty > Student) IMPLEMENTATION
Request added: Alice (student)
Request added: Bob (faculty)
Request added: Charlie (student)
Processing request: Bob (faculty)
Processing request: Alice (student)
Processing request: Charlie (student)
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Expected Outcome

- **Functional stack-based ticket management system.**
- **Clear demonstration of LIFO behavior.**

PROMPT:

#Generate a python code to build an emergency Help Desk system using Stack (LIFO)

#Implement stack operations:push(ticket), pop(), peek(),Add methods to check,is_empty(),is_full() (optional with size limit)

#Simulate at least 5 tickets being raised and resolved,Show LIFO behavior clearly,Print actions while pushing and popping

CODE:

```python
class HelpDeskStack:
    def __init__(self, size_limit=10):
        self.stack = []
        self.size_limit = size_limit

    def push(self, ticket):
        if len(self.stack) >= self.size_limit:
            print("Help Desk is full. Cannot add more tickets.")
            return
        self.stack.append(ticket)
        print(f"Ticket added: {ticket}")

    def pop(self):
        if self.is_empty():
            print("No tickets to resolve.")
            return None
        ticket = self.stack.pop()
        print(f"Resolving ticket: {ticket}")
        return ticket

    def peek(self):
        if self.is_empty():
            print("No tickets in the stack.")
            return None
        print(f"Next ticket to resolve: {self.stack[-1]}")
        return self.stack[-1]
```

```python
    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) >= self.size_limit
# Simulating the Help Desk system
help_desk = HelpDeskStack(size_limit=5)
help_desk.push("Ticket 1: Computer not working")
help_desk.push("Ticket 2: Internet connectivity issue")
help_desk.push("Ticket 3: Software installation request")
help_desk.push("Ticket 4: Printer not responding")
help_desk.push("Ticket 5: Email access problem")
help_desk.push("Ticket 6: Cannot add this ticket, stack is full")  # This should show a message that the stack is full
help_desk.peek()  # Should show the last ticket added (Ticket 5)
help_desk.pop()  # Resolves Ticket 5
help_desk.pop()  # Resolves Ticket 4
help_desk.pop()  # Resolves Ticket 3
help_desk.pop()  # Resolves Ticket 2
help_desk.pop()  # Resolves Ticket 1
help_desk.pop()  # This should show a message that there are no tickets to resolve
```

OUTPUT:

```
11.3.py
Ticket added: Ticket 3: Software installation request
Ticket added: Ticket 4: Printer not responding
Ticket added: Ticket 5: Email access problem
Help Desk is full. Cannot add more tickets.
Next ticket to resolve: Ticket 5: Email access problem
Resolving ticket: Ticket 5: Email access problem
Resolving ticket: Ticket 4: Printer not responding
Resolving ticket: Ticket 3: Software installation request
Resolving ticket: Ticket 2: Internet connectivity issue
Resolving ticket: Ticket 1: Computer not working
No tickets to resolve.
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

- **Insert**
- **Search**
- **Delete**

Starter Code

class HashTable:

pass

Expected Outcome

- **Collision handling using chaining**
- **Well-commented methods**

Prompt:

#Generate a python code to build a Hash Table with collision handling using chaining

insert(key, value),search(key),delete(key)

#Use chaining (list inside each bucket) to handle collisions,Add comments for clarity,Test with sample inputs

CODE:

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining

    def _hash(self, key):
        return hash(key) % self.size  # Simple hash function to determine bucket index

    def insert(self, key, value):
        index = self._hash(key)
        # Check if the key already exists in the bucket
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                print(f"Updated key: {key} with value: {value}")
                return
        # If key does not exist, add new key-value pair
        self.table[index].append((key, value))
        print(f"Inserted key: {key} with value: {value}")

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                print(f"Found key: {key} with value: {v}")
                return v
        print(f"Key: {key} not found")
        return None
```

```python
    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair from the bucket
                print(f"Deleted key: {key}")
                return
        print(f"Key: {key} not found for deletion")
# Testing the Hash Table with chaining
hash_table = HashTable(size=5)  # Create a hash table with 5 buckets
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
hash_table.insert("city", "New York")
hash_table.search("name")  # Should find Alice
hash_table.search("age")   # Should find 30
hash_table.search("country")  # Should not find this key
hash_table.delete("age")  # Should delete the key "age"
hash_table.search("age")   # Should not find "age" after deletion
```

OUTPUT:

```
'c:\Users\prash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
11.3.py'
Inserted key: name with value: Alice
Inserted key: age with value: 30
Inserted key: city with value: New York
Found key: name with value: Alice
Found key: age with value: 30
Key: country not found
Deleted key: age
Key: age not found
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

• Mapping table: Feature → Data Structure → Justification

• One fully working Python implementation

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

```python
# Generate a python code to design a Campus Resource Management System
# 1.Map features to suitable data structures:
# Student Attendance Tracking,Event Registration System,Library Book Borrowing,Bus Scheduling System,Cafeteria Order Queue
# 2.Provide justification (2-3 sentences each)
# 3.Implement one feature using Python
# 1. Student Attendance Tracking: Use a Hash Table to store student IDs and their attendance records. This allows for O(1) average time complexity f
# 2. Event Registration System: Use a Queue to manage event registrations. This ensures that registrations are processed in the order they were rece
# 3. Library Book Borrowing: Use a Linked List to manage the list of borrowed books. This allows for dynamic memory allocation and easy insertion/de
# 4. Bus Scheduling System: Use a Priority Queue to manage bus schedules. This allows for prioritizing certain routes or times based on demand, ensu
# 5. Cafeteria Order Queue: Use a Stack to manage cafeteria orders. This allows for a LIFO approach where the most recent orders are processed first
# Implementing the Event Registration System using a Queue
class EventRegistrationQueue:
    def __init__(self):
        self.queue = []

    def register(self, name):
        self.queue.append(name)
        print(f"{name} has registered for the event.")

    def process_registration(self):
        if not self.queue:
            print("No registrations to process.")
            return None
        name = self.queue.pop(0)
        print(f"Processing registration for {name}.")
        return name
# Testing the Event Registration System
event_queue = EventRegistrationQueue()
event_queue.register("Alice")
event_queue.register("Bob")
event_queue.register("Charlie")
event_queue.process_registration()  # Should process Alice first
event_queue.process_registration()  # Should process Bob next
event_queue.process_registration()  # Should process Charlie last
event_queue.process_registration()  # Should show no registrations to process
```

OUTPUT:

```
'c:\Users\prash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\de
11.3.py'
Alice has registered for the event.
Bob has registered for the event.
Charlie has registered for the event.
Processing registration for Alice.
Processing registration for Bob.
Processing registration for Charlie.
No registrations to process.
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```